

Софтуерен процес - дейностите нужни за разработване на софтуерни системи в срок и с високо качество

Софтуерен продукт - смята се системата, която се предоставя на клиента код + документация + данни

Целите на добрия процес са: ефикасност, поуверливост, преискузукост, качество.

Основни фази на софтуерния процес

1. Коммуникация - какво иска клиентът
2. Планиране - оценка, график
3. Моделиране - проектиране
4. Конструирование - кодиране + тестване
5. Внедряване - установка, поуверливост

Модел на водопада

Линейен процес при който всяка фаза завършва изцяло преди да започне следващата. Плюсове са, че е ясен и лесен за разбиране. Минуси са, че клиентът рядко знае/дава всички изисквания в началото, не е гъвкав при промени. Подходящ е ако изискванията са стабилни и ясни

Модел на бързата разработка

Компонентно-базиран, паралелен и с кратки времеви рамки. Подходящ е за проекти с добре разбирани изисквания и възможност за пренасочване на компоненти

Еволюционен модел

1. Итерационен - функционалността се дели на части и приоритетните части се правят първи
2. Итеративен - версиите се усъвършенстват цикъл след цикъл

Плюсове: клиентът използва системата преди да е завършена, най-големите грешки се тестват най-много.

Минуси: изисква активно участие на клиента

Прототипен модел

Прави се бърз прототип, за да се изяснят изискванията. Има два вида:

- изхвърляем - хвърля се след изяснение
- еволюционен - прототипът прераста в завършения продукт

Спираловиден модел - еволюционен модел, който съчетава прототипа и водопада. Всеки завой на спиралата минава през:

комуникация → планиране → моделиране → конструиране → внедряване.

Сравнителна характеристика

модел	гъвкавост	Упр. на риска	Участие клиент	кога е подходящ
водопад	Ниска	Слабо	В началото	стабилни и ясни изисквания
бърза разр.	Средна	Слабо	Умерено	кратки срокове
еволюционен	Висока	Средно	Постоянно	менящи се изисквания
прототипен	Висока	Средно	Високо	неясни изисквания
спираловиден	Висока	Силно	Постоянно	големи, рисковани проекти

Гъвкави методи

- Extreme programming - доброволно добрите практики за писане на код до крайност. Като такива са:
- test-driven development
- pair programming
- клиентът е част от екипа
- малки и чести релизи
- прост код

• SCRUM

За разлика от XP е фокусиран върху планиране и управление. Има списък със задачи (product backlog). Взима се част от този backlog и се прави за 2-4 седмици (sprint), като всяка сутрин има кратка среща (daily scrum), а в края на спринта се прави преглед на направеното (sprint review) и това се повтаря.

Изисквания

- потребителски изисквания - на високо ниво, на езика на клиента
- системни изисквания - технически, основа за дизайна
- функционални изисквания - какво трябва да прави системата: услуги, поведение, реакция при вхор
- нефункционални изисквания - как работи системата, какви ограничения и качества има: производителност, сигурност, надеждност. Има три вида:
 - изисквания към продукта
 - организационни (процес на разработка)
 - външни (корпоративни, етични)

Анализ и проектиране на софтуерни изисквания

Разработката на изисквания минава през:

осъществимост → анализ → специфициране → валидиране

UML е стандартен език за моделиране. Има две големи групи учители:

- структурни - Class, Object и др.
- поведенически - Use case, Sequence и др.

Class - описва класовете, атрибутите им и връзките между тях

Use case - взаимоотношенията на системата с актьорите в конкретен случай

sequence - обектите и съобщенията между тях, подредени хронологично

Верификация и валидация

- Верификация - съответства ли проектът на изискванията
- валидация - това ли иска клиентът наистина

Управление на качеството гарантира че продуктът отговаря на стандартите и изискванията. Особено важно е при големи и сложни системи. Качеството се гарантира чрез добър процес и чрез постоянни проверки